Deep Neural Networks - Programming Assignment 2

Student: Souvik Dutta | ID: 2026AJ05991

Configuration: h=64; d=128; four heads; seeds 42,43,44. Individual ablation: RNN gradient clipping at T=200.

Methods: UCI HAR uses nine channels, 128 steps and six activities with subject-wise splitting and training-only normalization. The adding task uses two markers and the target sum. Scratch recurrent cells and Transformer blocks were checked against framework/reference calculations.

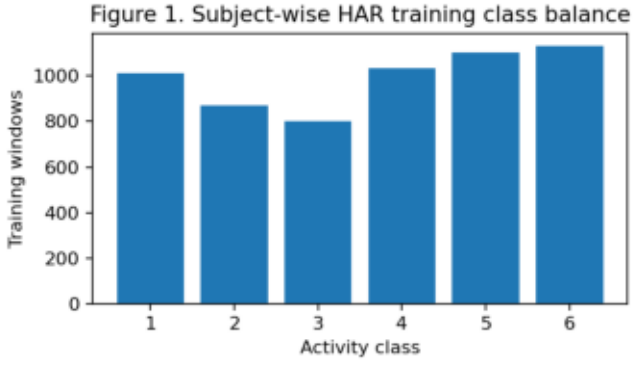
Execution: Full experiments ran on a Colab Tesla T4 with PyTorch 2.14.1+cu130. Problem 1 took approximately 274.68 seconds and Problem 2 approximately 297.92 seconds, excluding downloads and startup.

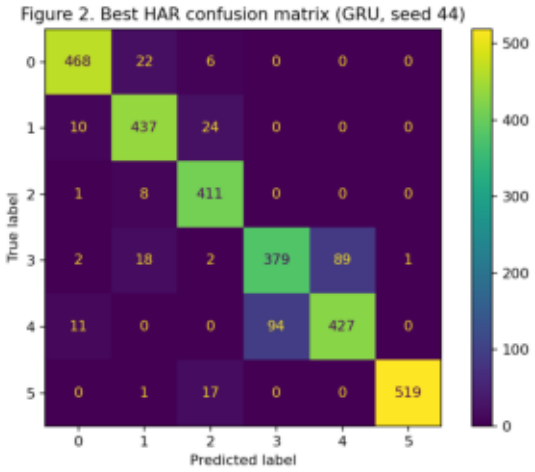
Problem 1 - HAR classification

Table 1. HAR results (mean +/- sample SD): RNN accuracy 65.739 +/- 3.335%, macro-F1 64.994 +/- 3.338%, 5,190 parameters, 0.186 s/epoch. LSTM: 85.997 +/- 2.952%, 85.664 +/- 3.108%, 19,590 parameters, 0.183 s/epoch. GRU: 88.847 +/- 0.674%, 88.838 +/- 0.731%, 14,790 parameters, 0.176 s/epoch.

GRU is the strongest wearable choice: best accuracy/F1, lower parameter count than LSTM, and lowest epoch time. Figure 1 is class balance; Figure 2 is the best-run confusion matrix.

For input size 9 and h=64, one-bias equation counts are RNN 4,736, LSTM 18,944 and GRU 14,208. PyTorch two-bias counts are 4,800, 19,200 and 14,400.





Problem 1 - Adding task

Table 2. Mean MSE over three seeds; solved means MSE < 0.05. T=50: RNN 0.17459, LSTM 0.17044, GRU 0.00965 (3/3 solved). T=100: RNN 0.16973, LSTM 0.16955, GRU 0.13319 (0/3). T=200: RNN 0.16812, LSTM 0.16833, GRU 0.16721 (0/3). T=400: RNN 0.15367, LSTM 0.15337, GRU 0.15264 (0/3).

Figure 3 gives MSE versus length. Figure 4 gives T=200 training/validation curves for seed 42. The short-sequence GRU learns marker positions, while long horizons expose recurrent credit-assignment difficulty under the fixed budget.

Figure 6. Adding-problem MSE against sequence length

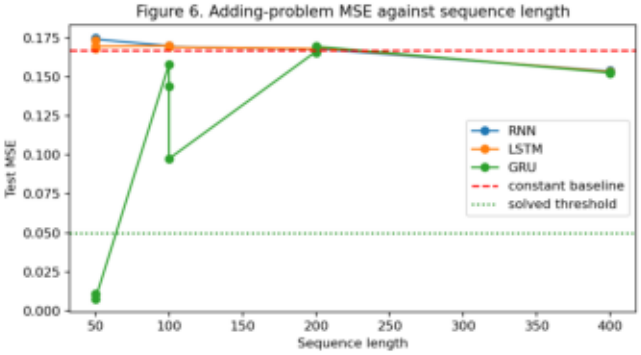
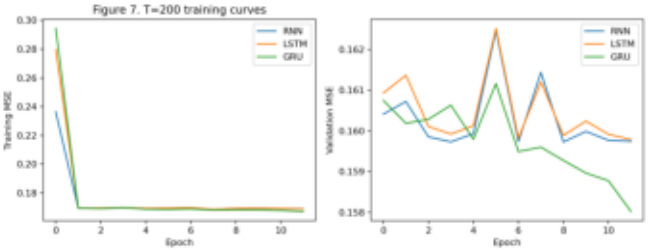


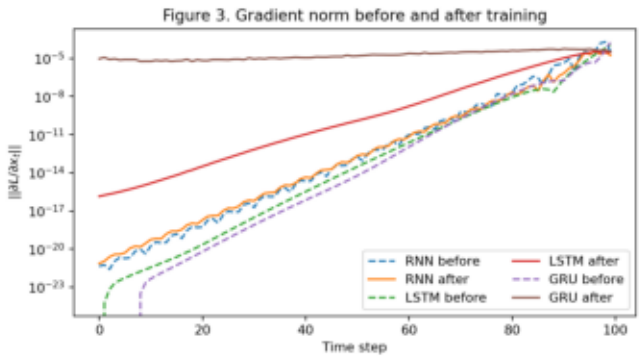
Figure 7. T=200 training curves

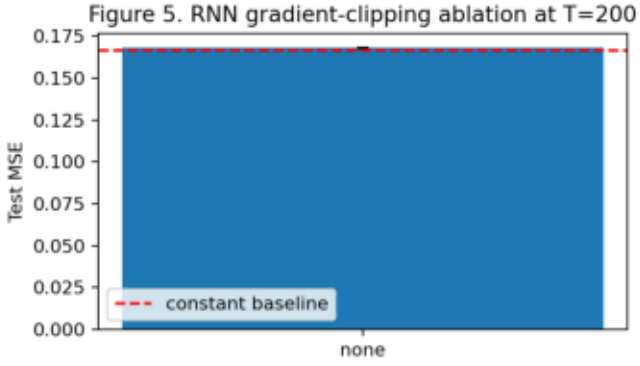


Problem 1 - Gradients, capacity and ablation

Figure 5 reports T=100 before/after gradient profiles; training mode is used for cuDNN backward without an optimizer step. Figure 6 is the hidden-size sweep at 16, 32, 64 and 128 over three seeds. At h=64, LSTM accuracy is 85.997% with 19,590 parameters; GRU is 88.847% with 14,790.

Figure 7: no clipping MSE 0.16812 +/- 0.00025; norm 1.0 MSE 0.16845 +/- 0.00045. Clipping did not improve this measured setup.

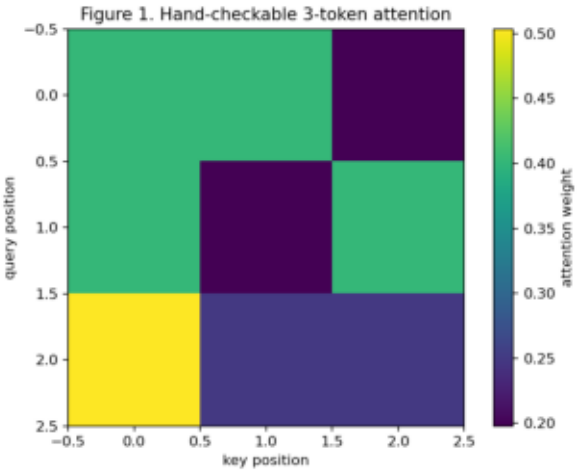


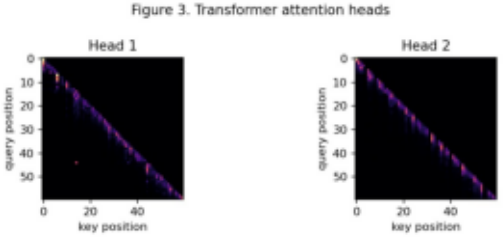


Problem 2 - Attention and Transformer HAR

Figure 8. Causal hand attention: Q=[[1,0],[0,1],[1,1]], K=[[1,1],[1,0],[0,1]], V=[[1,2],[3,4],[5,6]]. With a=1/sqrt(2)=0.707107, scores are [[a,a,0],[a,0,a],[2a,a,a]]. Softmax rows are [0.4011,0.4011,0.1978], [0.4011,0.1978,0.4011], and [0.5035,0.2483,0.2483]. Multiplication by V gives [[2.593327,3.593327],[3,4],[2.489531,3.489531]], matching code and PyTorch. The displayed heatmap is causal: all upper-triangular future positions are black.

Table 3. Transformer HAR: None 89.164 +/- 1.163% accuracy and 89.068 +/- 1.134% F1; sinusoidal 88.214 +/- 0.680% and 88.088 +/- 0.734%; learned 88.542 +/- 1.196% and 88.368 +/- 1.241%. Each has 415,494 parameters.





Problem 2 - Language model, ablations and conclusion

Table 4. Causal LM: context 128, width 128, four heads, four layers. Final T4 validation loss is 1.5990 nats = 2.3069 bits per character; the model has 810,049 parameters. Samples at 0.2, 0.8 and 1.5 are saved.

The heads ablation reports 215,233 because it uses one block; the LM uses four. For d=128 and d_ff=512, encoder weights are 4d? + 2*d*d_ff = 196,608 before biases and LayerNorm.

Table 5. Context 64/128/256/512 recorded causal/unmasked timings and peak memory: (0.001488,0.001084,18.93 MB), (0.001327,0.001055,20.14 MB), (0.001474,0.001166,24.37 MB), (0.001569,0.001551,39.25 MB). This follows O(n?d); unmasked attention is only an ablation.

AI assistance supported structuring, debugging, tests, plotting and report organisation. All numerical results are from the author's T4 runs. References: Hochreiter & Schmidhuber; Cho et al.; Vaswani et al.; Anguita et al.; Karpathy Tiny Shakespeare; PyTorch documentation.

Figure 2. Tiny Shakespeare validation learning curve

